2. The assignment operators is the only operator that is not inherited. The reason for this is easy to understand. Since a derived class will contain members not found in the base class, the overloaded = relative to the base has no knowledge of the members added by the derived class and, as such, cannot properly copy those new members.

REVIEW SKILLS CHECK: Chapter 8

1.
```
#include <iostream>
using namespace std;

class airship
{
protected:
    int passengers;
    double cargo;
};

class airplane : public airship
{
    char engine; // p for propeller, j for jet
    double range;
public:
    airplane(int p, double c, char e, double r)
    {
        passengers = p;
        cargo = c;
        engine = e;
        range = r;
    }
    void show();
};

class balloon : public airship
{
    char gas; // h for hydrogen, e for helium
    double altitude;
public:
    balloon(int p, double c, char g, double a)
    {
        passengers = p;
        cargo = c;
        gas = g;
        altitude = a;
    }
    void show();
};

void airplane::show()
{
    cout << "Passengers: " << passengers << '\n';
    cout << "Cargo capacity: " << cargo << '\n';
```

```
    cout << "Engine: ";
    if(engine=='p')
        cout << "Propeller\n";
    else
        cout << "Jet\n";
    cout << "Range: " << range << '\n';
}

void balloon::show()
{
    cout << "Passengers: " << passengers << '\n';
    cout << "Cargo capacity: " << cargo << '\n';
    cout << "Gas: ";
    if(gas=='h')
        cout << "Hydrogen\n";
    else
        cout << "Helium\n";
    cout << "Altitude: " << altitude << '\n';
}

int main()
{
    balloon b(2, 500.0, 'h', 12000.0);
    airplane b727(100, 40000.0, 'j', 40000.0);

    b.show();
    cout << '\n';
    b727.show();

    return 0;
}
```

2. The **protected** access specifier causes a class member to be private to its class but still accessible by any derived class.

3. The program displays the following output, which indicates when the constructors and destructors are called.

```
Constructing A
Constructing B
Constructing C
Destructing C
Destructing B
Destructing A
```

4. Constructors are called in the order ABC, destructors in the order CBA.

5.
```
#include <iostream>
using namespace std;
```

```
class base
{
    int i, j;
public:
    base(int x, int y) { i = x; j = y; }
    void showij() { cout << i << ' ' << j << '\n'; }
};

class derived : public base
{
    int k;
public:
    derived(int a, int b, int c) : base(b, c)
    {
        k = a;
    }
    void show() { cout << k << ' '; showij(); }
};

int main()
{
    derived ob(1, 2, 3);

    ob.show();

    return 0;
}
```

6. The missing words are "general" and "specific".

8.2 EXERCISES

1. ```
   #include <iostream>
   using namespace std;

   int main()
   {
       cout.setf(ios::showpos);

       cout << -10 << ' ' << 10 << '\n';

       return 0;
   }
   ```

2. ```
   #include <iostream>
   using namespace std;

   int main()
   {
       cout.setf(ios::showpoint | ios::uppercase |
   ```

```
                ios::scientific);

    cout << 100.0;

    return 0;
}
```

3. ```
#include <iostream>
using namespace std;

int main()
{
    ios::fmtflags f;

    f = cout.flags(); // store flags

    cout.unsetf(ios::dec);
    cout.setf(ios::showbase | ios::hex);
    cout << 100 << '\n';

    cout.flags(f); // reset flags

    return 0;
}
```

## 8.3 EXERCISES

1. ```
// Create a table of log10 and log from 2 through 100.
#include <iostream>
#include <cmath>
using namespace std;

int main()
{
    double x;

    cout.precision(5);
    cout << "          x       log x        ln e\n\n";

    for(x = 2.0; x <= 100.0; x++)
    {
        cout.width(10);
        cout << x << ' ';
        cout.width(10);
        cout << log10(10) << ' ';
        cout.width(10);
        cout << log(x) << '\n';
    }

    return 0;
}
```

2.
```
#include <iostream>
#include <cstring>
using namespace std;

void center(char *s);

int main()
{
     center("Hi there!");
     center("I like C++.");

     return 0;
}

void center(char *s)
{
     int len;

     len = 40+(strlen(s)/2);

     cout.width(len);
     cout << s << '\n';
}
```

1a.
```
// Create a table of log10 and log from 2 through 100.
#include <iostream>
#include <iomanip>
#include <cmath>
using namespace std;

int main()
{
     double x;

     cout.precision(5);
     cout << "          x       log x        ln e\n\n";

     for(x = 2.0; x <= 100.0; x++)
     {
          cout << setw(10) << x << ' ';
          cout << setw(10) << log10(10) << ' ';
          cout << setw(10) << log(x) << '\n';
     }

     return 0;
}
```

1b.
```
#include <iostream>
#include <iomanip>
#include <cstring>
using namespace std;
```

```
void center(char *s);

int main()
{
    center("Hi there!");
    center("I like C++.");

    return 0;
}

void center(char *s)
{
    int len;

    len = 40+(strlen(s)/2);

    cout << setw(len) << s << '\n';
}
```

2. `cout << setiosflags(ios::showbase | ios::hex) << 100;`

3. Setting the **boolalpha** flag on an output stream causes Boolean values to be displayed using the words *true* and *false*. Setting **boolalpha** on an input stream allows you to enter Boolean values using the words *true* and *false*.

8.5 EXERCISES

1. ```
   #include <iostream>
   #include <cstring>
   #include <cstdlib>
   using namespace std;

   class strtype
   {
       char *p;
       int len;
   public :
       strtype(char * ptr);
       ~strtype() { delete [] p; }
       friend ostream &operator <<(ostream &stream, strtype &ob)
          ;
   };

   strtype::strtype(char *ptr)
   {
       len = strlen(ptr)+1;
       p = new char(len);
       if(!p)
       {
           cout << "Allocation error \n";
           exit (1) ;
   ```
   ```

```
    }
    strcpy (p, ptr );
}

ostream &operator<<(ostream &stream, strtype &ob)
{
    stream << ob.p;

    return stream;
}

int main()
{
    strtype s1("This is a test."), s2("I like C++.");
    cout << s1 << '\n' << s2 << '\n';

    return 0;
}
```

2.
```
#include <iostream>
using namespace std;

class planet
{
protected:
    double distance; // miles from the sun
    int revolve;     // in days
public:
    planet(double d, int r) { distance = d; revolve = r; }
};

class earth : public planet
{
    double circumference; // circumference of orbit
public:
    earth(double d, int r) : planet(d, r)
    {
        circumference = 2*distance*3.1416;
    }
    friend ostream &operator<<(ostream &stream, earth ob);
};

ostream &operator<<(ostream &stream, earth ob)
{
    stream << "Distance from sun: " << ob.distance << '\n';
    stream << "Days in orbit: " << ob.revolve << '\n';
    stream << "Circumference of orbit: ";
    stream << ob.circumference << '\n';
}

int main()
```

```
{
    earth ob(93000000, 365);

    cout << ob;

    return 0;
}
```

3. An inserter cannot be a member function because the object that generates a call to the inserter is *not* an object of a user-defined class.

8.6 EXERCISES

1.
```
#include <iostream>
#include <cstring>
#include <cstdlib>
using namespace std;

class strtype
{
    char *p;
    int len;
public :
    strtype(char *ptr);
    ~strtype() { delete [] p; }
    friend ostream &operator <<(ostream &stream, strtype &ob)
       ;
    friend istream &operator >>(istream &stream, strtype &ob)
       ;
};

strtype::strtype(char *ptr)
{
    len = strlen(ptr)+1;
    p = new char(len);
    if(!p)
    {
        cout << "Allocation error \n";
        exit (1) ;
    }
    strcpy (p, ptr );
}

ostream &operator<<(ostream &stream, strtype &ob)
{
    stream << ob.p;

    return stream;
}

istream &operator>>(istream &stream, strtype &ob)
```

```
{
    char temp[255];

    stream >> temp;

    if(strlen(temp) >= ob.len)
    {
        delete [] ob.p;
        ob.len = strlen(temp)+1;
        ob.p = new char(ob.len);
        if(!ob.p)
        {
            cout << "Allocation error\n";
            exit(1);
        }
    }
    strcpy(ob.p, temp);

    return stream;
}

int main()
{
    strtype s1("This is a test."), s2("I like C++.");
    cout << s1 << '\n' << s2 << '\n';

    cout << "\nEnter a string: ";
    cin >> s1;
    cout << s1;

    return 0;
}
```

2.
```
#include <iostream>
using namespace std;

class factor
{
    int num; // number
    int lfact; // lowest factor
public:
    factor(int i);
    friend ostream &operator<<(ostream &stream, factor ob);
    friend istream &operator>>(istream &stream, factor &ob);
};

factor::factor(int i)
{
    int n;

    num = i;
```

```
    for(n=2; n<(i/2); n++)
        if(!(i%n))
            break;

    if(n<(i/2))
        lfact = n;
    else
        lfact = 1;
}

istream &operator>>(istream &stream, factor &ob)
{
    stream >> ob.num;

    int n;

    for(n=2; n<(ob.num/2); n++)
        if(!(ob.num%n))
            break;

    if(n<(ob.num/2))
        ob.lfact = n;
    else
        ob.lfact = 1;

    return stream;
}

ostream &operator<<(ostream &stream, factor ob)
{
    stream << ob.lfact << " is lowest factor of ";
    stream << ob.num << '\n';

    return stream;
}

int main()
{
    factor o(32);

    cout << o;

    cin >> o;
    cout << o;

    return 0;
}
```

MASTERY SKILLS CHECK: Chapter 8

1.
```cpp
#include <iostream>
using namespace std;

int main()
{
    cout << 100 << ' ';

    cout.unsetf(ios::dec); // clear dec flag
    cout.setf(ios::hex);
    cout << 100 << ' ';

    cout.unsetf(ios::hex); // clear hex flag
    cout.setf(ios::oct);
    cout << 100 << '\n';

    return 0;
}
```

2.
```cpp
#include <iostream>
using namespace std;

int main()
{
    cout.setf(ios::left);
    cout.precision(2);
    cout.fill('*');
    cout.width(20);

    cout << 1000.5364 << '\n';

    return 0;
}
```

3a.
```cpp
#include <iostream>
using namespace std;

int main()
{
    cout << 100 << ' ';

    cout << hex << 100 << ' ';

    cout << oct << 100 << '\n';

    return 0;
}
```

3b.
```cpp
#include <iostream>
#include <iomanip>
using namespace std;
```

```
int main()
{
    cout << setiosflags(ios::left);
    cout << setprecision(2);
    cout << setfill('*');
    cout << setw(20);

    cout << 1000.5364 << '\n';

    return 0;
}
```

4.
```
ios::fmtflags f;

f = cout.flag(); // save

// ...

cout.flags(f); // restore
```

5.
```
#include <iostream>
using namespace std;

class pwr
{
    int base;
    int exponent;
    double result; // base to the exponent power
public:
    pwr(int b, int e);
    friend ostream &operator<<(ostream &stream, pwr ob);
    friend istream &operator>>(istream &stream, pwr &ob);
};

pwr::pwr(int b, int e)
{
    base = b;
    exponent = e;

    result = 1;
    for( ; e; e--)
        result = result * base;
}

ostream &operator<<(ostream &stream, pwr ob)
{
    stream << ob.base << "^" << ob.exponent;
    stream << " is " << ob.result << '\n';
}
istream &operator>>(istream &stream, pwr &ob)
{
```

```
    int b, e;

    cout << "Enter base and exponent: ";
    stream >> b >> e;

    pwr temp(b, e); // create temporary

    ob = temp;

    return stream;
}

int main()
{
    pwr ob(10, 2);

    cout << ob;

    cin >> ob;

    cout << ob;

    return 0;
}
```

6.
```
// This program draws boxes.
#include <iostream>
using namespace std;

class box
{
    int len;
public:
    box(int l) { len = l; }
    friend ostream &operator<<(ostream &stream, box ob);
};

// Draw a box.
ostream &operator<<(ostream &stream, box ob)
{
    int i, j;

    for(i=0; i<ob.len; i++)
        stream << '*';
    stream << '\n';
    for(i=0; i<ob.len-2;i++)
    {
        stream << '*';
        for(j=0; j<ob.len-2;j++)
            stream << ' ';
        stream << "*\n";
```

```
    }
    for(i=0; i<ob.len; i++)
        stream << '*';
    stream << '\n';

    return stream;
}

int main()
{
    box b1(4), b2(7);

    cout << b1 << endl << b2;

    return 0;
}
```

CUMULATIVE SKILLS CHECK: Chapter 8

1.
```
#include <iostream>
using namespace std;

#define SIZE 10

// Declare a stack class for characters
class stack
{
    char stck[SIZE]; // holds the stack
    int tos; // index of top of stack
public:
    stack();
    void push(char ch); // push character on stack
    char pop(); // pop character from stack
    friend ostream &operator<<(ostream &stream, stack ob);
};

// Initialize the stack
stack::stack()
{
    tos = 0;
}

// Push a character.
void stack::push(char ch)
{
    if(tos==SIZE)
    {
        cout << "Stack is full\n";
        return;
    }
    stck[tos] = ch;
```

```
    tos++;
}

// Pop a character.
char stack::pop()
{
    if(tos==0)
    {
        cout << "Stack is empty\n";
        return 0; // return null on empty stack
    }
    tos--;
    return stck[tos];
}

ostream &operator<<(ostream &stream, stack ob)
{
    char ch;

    while(ch=ob.pop())
        stream << ch;
    stream << endl;

    return stream;
}

int main()
{
    stack s;

    s.push('a');
    s.push('b');
    s.push('c');

    cout << s;
    cout << s;

    return 0;
}
```

2.
```
#include <iostream>
#include <ctime>
using namespace std;

class watch
{
    time_t t;
public:
    watch() { t = time(NULL); }
    friend ostream &operator<<(ostream &stream, watch ob);
};
```

```
ostream &operator<<(ostream &stream, watch ob)
{
    struct tm *localt;

    localt = localtime(&ob.t);
    stream << asctime(localt) << endl;

    return stream;
}

int main()
{
    watch w;

    cout << w;

    return 0;
}
```

3.
```
#include <iostream>
using namespace std;

class ft_to_inches
{
    double feet;
    double inches;
public:
    void set(double f)
    {
        feet = f;
        inches = f * 12;
    }
    friend istream &operator>>(istream &stream,
                               ft_to_inches &ob);
    friend ostream &operator<<(ostream &stream,
                               ft_to_inches ob);
};

istream &operator>>(istream &stream, ft_to_inches &ob)
{
    double f;

    cout << "Enter feet: ";
    stream >> f;
    ob.set(f);

    return stream;
}

ostream &operator<<(ostream &stream, ft_to_inches ob)
```

```
{
    stream << ob.feet << " feet is " << ob.inches;
    stream << " inches\n";

    return stream;
}

int main()
{
    ft_to_inches x;

    cin >> x;
    cout << x;

    return 0;
}
```

REVIEW SKILLS CHECK: Chapter 9

1.
```
#include <iostream>
using namespace std;

int main()
{
    cout.width(40);
    cout.fill(':');

    cout << "C++ is fun" << '\n';

    return 0;
}
```

2.
```
#include <iostream>
using namespace std;

int main()
{
    cout.precision(4);
    cout << 10.0/3.0 << '\n';

    return 0;
}
```

3.
```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    cout << setprecision(4) << 10.0/3.0 << '\n';
```